EPD 05 · Ej3 — Carga perezosa de imágenes

Programación Avanzada · Grado en Ingeniería Informática · Curso 2026/2027

1. Objetivo

Medir el tiempo de carga de una página con muchas imágenes pesadas y comparar tres estrategias: **sin carga perezosa**, carga perezosa con el **plugin LazyLoad de jQuery** (`jquery.lazyload 1.9.7`) y carga perezosa **nativa de HTML5** (`loading="lazy"`). Se presta especial atención a la métrica **TTFB** (*Time To First Byte*), según la guía de Kinsta.

2. Páginas de prueba

Las tres páginas son idénticas salvo en cómo se cargan las imágenes. Cada una contiene 12 imágenes JPEG de 1920×1280 px (~720 KB cada una, **8,8 MB** en total), mostradas a un ancho máximo de 960 px, con texto entre ellas para que la mayoría queden fuera de la zona visible al abrir la página.

Archivo	Estrategia	Marcado de cada imagen
`sin-lazy.html`	Sin carga perezosa	`<img src="img/foto01.jpg">`
`lazyload-plugin.html`	Plugin jQuery LazyLoad	`<img class="lazy" src="img/placeholder.jpg" data-original="img/foto01.jpg">`
`lazy-html5.html`	HTML5 nativo	`<img src="img/foto01.jpg" loading="lazy">`

Integración del plugin (tras cargar jQuery 3.7.1 y `jquery.lazyload.js`):

```
$(document).ready(function () {
    $("img.lazy").lazyload({
        threshold: 200,   // empieza a cargar 200 px antes de que la imagen sea visible
        effect: "fadeIn"  // aparición suave
    });
});
```

El plugin sustituye el `src` por el valor de `data-original` cuando la imagen se acerca a la zona visible. Mientras tanto se muestra `placeholder.jpg` (630 bytes). Los atributos `width` / `height` reservan el hueco de cada imagen y evitan saltos de diseño.



Figura 1. Página con el plugin LazyLoad: solo se ha descargado la primera fotografía.

3. Procedimiento de medición en el navegador

1. Servir la carpeta del ejercicio con un servidor HTTP local (no abrir con `file://`), por ejemplo: `python -m http.server 8000` y abrir `http://localhost:8000/`.
2. Abrir las herramientas de desarrollador (**F12**) → pestaña **Red / Network**.
3. Marcar **Desactivar caché / Disable cache** para que cada recarga descargue todo de nuevo.
4. Opcional: en el desplegable de limitación (*No throttling*) elegir un perfil de red (p. ej. *Fast 4G*) para simular una conexión real.
5. Recargar la página (**Ctrl+Shift+R**) y anotar los valores de la barra inferior: número de peticiones, datos transferidos, **DOMContentLoaded** y **Load**.
6. Para el **TTFB**: seleccionar la petición del documento HTML → pestaña **Timing / Tiempos** → valor **Waiting for server response (TTFB)**.
7. Desplazarse hasta el final de la página y observar cómo aparecen nuevas peticiones de imágenes en las variantes perezosas.
8. Repetir cada medición varias veces y quedarse con la mediana.

4. Resultados

Entorno: Chromium (Playwright), ventana 1366×768, caché desactivada, servidor HTTP local, mediana de 5 ejecuciones por caso. Se midió sin limitación de red y con una red simulada de tipo 4G (9 Mbps de bajada, 60 ms de latencia).

4.1 Sin limitación de red (localhost)

Métrica	Sin lazy	Plugin LazyLoad	HTML5 loading="lazy"
TTFB del documento	1,0 ms	1,0 ms	0,9 ms
DOMContentLoaded	23 ms	40 ms	17 ms
Evento load	97 ms	40 ms	17 ms
Peticiones hasta load	14	5	2
Datos transferidos hasta load	8,82 MB	107 KB	8 KB
Imágenes descargadas al abrir (sin desplazarse)	12	1	5
Datos tras recorrer toda la página	8,82 MB	8,92 MB	8,82 MB

4.2 Red 4G simulada (9 Mbps, 60 ms)

Métrica	Sin lazy	Plugin LazyLoad	HTML5 loading="lazy"
TTFB del documento	0,9 ms	0,9 ms	0,9 ms
DOMContentLoaded	86 ms	241 ms	79 ms
Evento load	**8 010 ms**	**241 ms**	**142 ms**
Imágenes descargadas al abrir	12	1	3

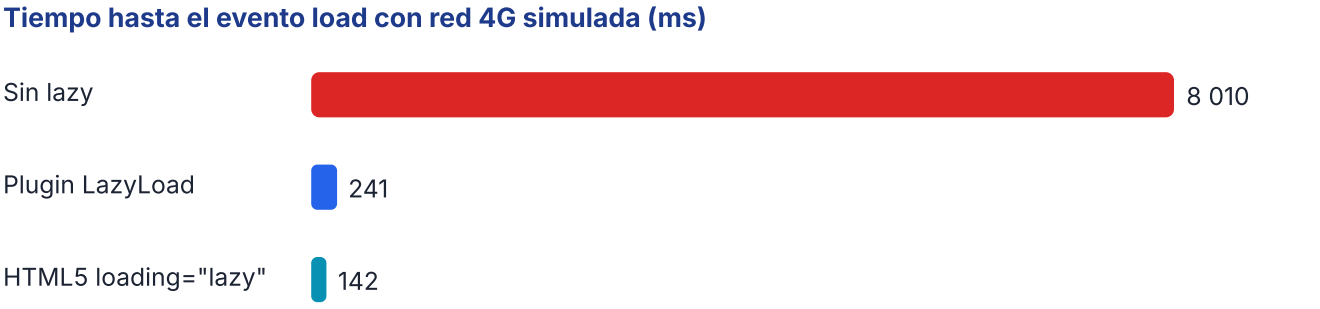


Figura 2. Con red 4G, la carga perezosa reduce el tiempo hasta *load* unas 33 veces (plugin) y 56 veces (HTML5).

5. Análisis

- **El TTFB no cambia** entre las tres variantes (~1 ms). El TTFB mide cuánto tarda el servidor en empezar a responder al documento HTML y depende del servidor, la red y la generación de la página, no de cómo se carguen después las imágenes. La carga perezosa mejora el **tiempo de carga total** y los datos transferidos, no el TTFB.
- **Sin carga perezosa**, el navegador descarga las 12 imágenes (8,8 MB) antes del evento *load*. Con una red 4G eso supone unos **8 segundos**.
- **Plugin LazyLoad:** al abrir solo se descarga la imagen visible. El evento *load* pasa de 8 s a 0,24 s. Su coste es que añade jQuery y el plugin (~100 KB) y que depende de JavaScript: sin JS las imágenes no se cargarían. Además, el *DOMContentLoaded* es algo más tardío por la descarga de estos scripts.

- **HTML5 `loading="lazy"`:** es la opción más ligera (sin scripts) y obtiene los mejores tiempos. El navegador decide el margen de anticipación (Chromium descarga por adelantado las imágenes situadas a unos 1 250–2 500 px de la zona visible), por eso al abrir descarga 3–5 imágenes en lugar de 1. Estas descargas no retrasan el evento *load*.
- Tras recorrer la página entera, las tres variantes acaban descargando prácticamente lo mismo. La ventaja de la carga perezosa está en que el usuario solo paga los datos de lo que realmente ve.

> **Fiabilidad.** Como advierte el enunciado, estas mediciones se han hecho con un servidor local: el TTFB y los tiempos sin limitación son irreales (casi 0 ms). Por eso se añadió la simulación de red 4G, que muestra con claridad el efecto de la carga perezosa. Para un TTFB realista hay que publicar la web en un servidor accesible desde Internet (apartado 7).

6. Conclusiones

La carga perezosa reduce drásticamente los datos y el tiempo de la carga inicial cuando hay imágenes fuera de la zona visible. Hoy en día la opción recomendable es el atributo nativo `loading="lazy"`, soportado por todos los navegadores modernos, más ligero y sin dependencias. El plugin de jQuery sigue siendo útil cuando se necesita más control (umbral exacto, efectos de aparición como `fadeIn`, eventos personalizados o compatibilidad con navegadores antiguos). Ninguna de las dos técnicas mejora el TTFB, que depende del servidor.

7. EXTRA: medición con NGROK

NGROK crea un túnel público HTTPS hacia un servidor local, de modo que la página se descarga a través de Internet y las mediciones (sobre todo el TTFB) son más realistas.

1. Crear una cuenta gratuita en `https://ngrok.com` y copiar el *authtoken* del panel.
2. Instalar ngrok (Windows: `winget install ngrok.ngrok`; macOS: `brew install ngrok`; Linux: paquete del sitio oficial).
3. Configurar el token: `ngrok config add-authtoken <TOKEN>`
4. En la carpeta del ejercicio, arrancar el servidor local: `python -m http.server 8000`
5. En otra terminal, abrir el túnel: `ngrok http 8000`. Copiar la URL pública que aparece en *Forwarding* (p. ej. `https://xxxx.ngrok-free.app`).
6. Abrir esa URL en el navegador. La primera vez ngrok muestra una página de aviso: pulsar *Visit Site*.
7. Repetir el procedimiento del apartado 3 para las tres páginas y anotar los resultados. El panel local `http://127.0.0.1:4040` de ngrok muestra además todas las peticiones recibidas con su duración.

Resultados con NGROK (rellenar con las mediciones propias):

Métrica	Sin lazy	Plugin LazyLoad	HTML5 loading="lazy"
TTFB del documento			
DOMContentLoaded			
Evento load			
Peticiones / datos transferidos			

Con NGROK cabe esperar un TTFB de decenas o cientos de milisegundos, porque cada petición viaja a los servidores de ngrok y vuelve al equipo local. Debería ser similar en las tres páginas. Las diferencias en el evento *load* serán todavía mayores que en local, porque descargar 8,8 MB por Internet es mucho más lento.